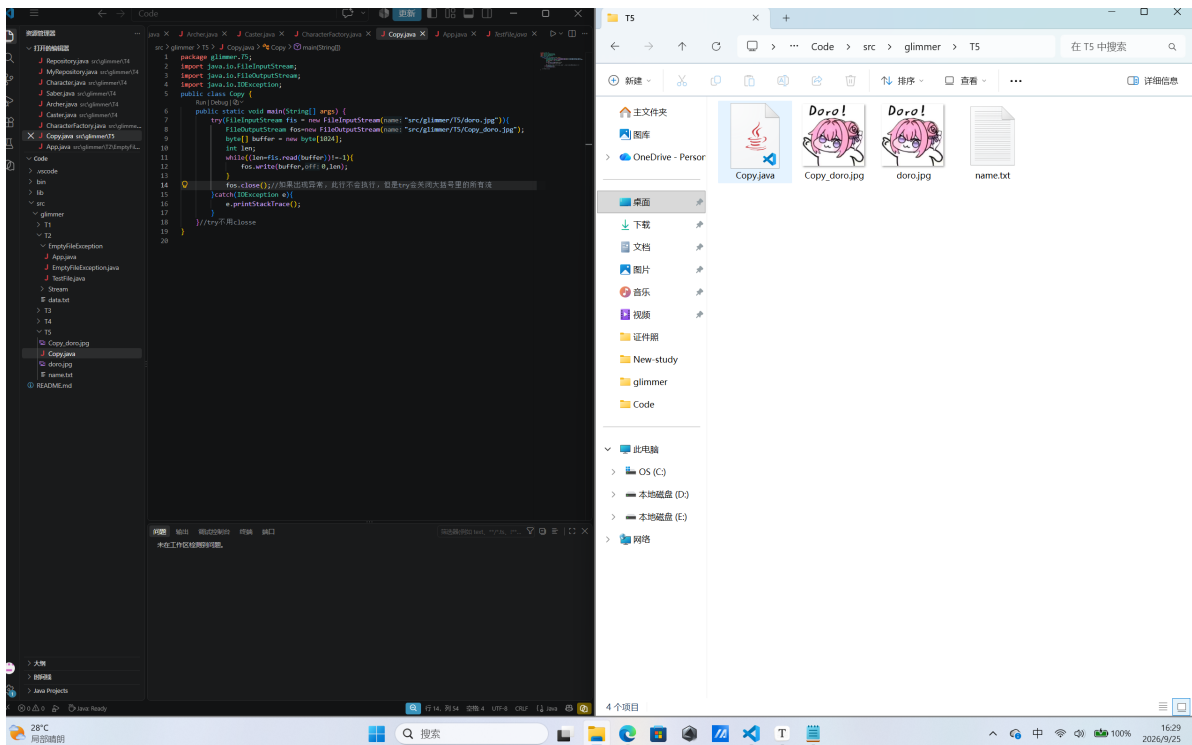
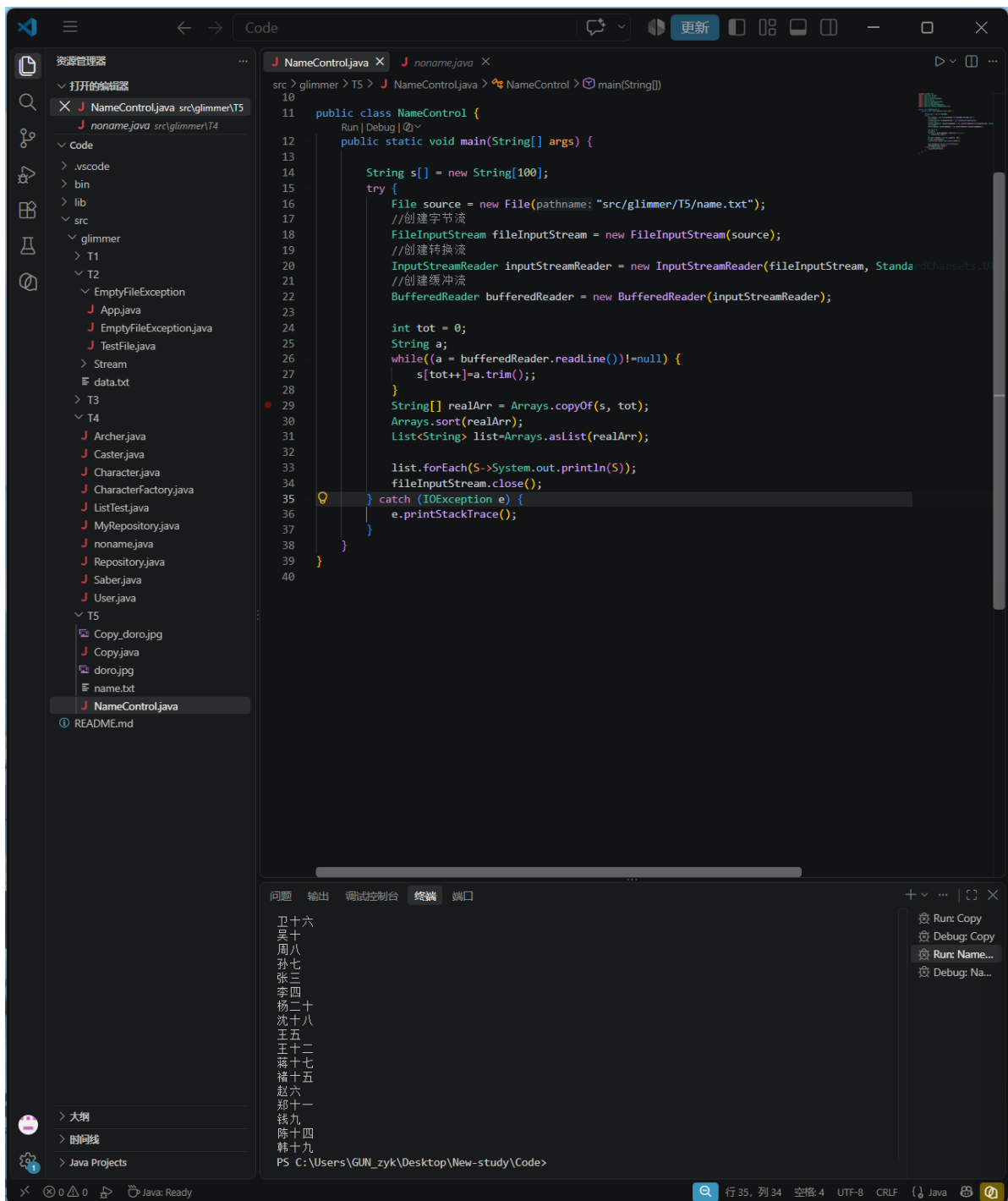


Q1



Q2 (转成stream+lambda语法输出)



maven

是什么

管理项目（把项目看成对象），

依赖管理（从中央仓库拿共享项目）

构建生命周期（用插件）

优点：标准化，跨平台

仓库

存资源，jar包

关于私服：1.速度快 2.版权

配置远程仓库，用的阿里云的镜像

坐标

找jar包

命名规范：

- groupId 隶属组织的名称
- artifactId 项目名称
- version 版本号
- packaging 打包方式

中央仓库会有坐标书写格式

命令

```
//编译
mvn compile
//清理
mvn clean
//测试
mvn test
//打包
mvn package
//安装到本地仓库
mvn install
```

生命周期

compile->target->test

项目结构

我先让AI完成了整个任务（很多东西完全陌生，所以我打算以解释代码的方式来学习）

这是一个**Java 原生 Http 服务端 + Http 客户端**的简易快递查询 Demo，**没有 SpringBoot**，用JDK 自带 `com.sun.net.httpserver` 实现 Web 服务。 结构：

```
com.express
├─ QueryRequest.java    // 客户端发给服务器的数据模型（请求体）
├─ QueryResponse.java   // 服务器返回给客户端的数据模型（响应体）
├─ Server.java          // Http服务端，监听端口，接收客户端POST请求
└─ Client.java          // Http客户端，输入单号手机号，POST请求服务器
```

1.先建立QueryRequest请求实体，private快递单号手机号

2.再建立QueryResponse响应实体，负责返回查询结果

客户端和服务端代码

代码里有注释

HTTP

HttpServer

方法	作用
<code>createContext("/path", handler)</code>	注册路径，把 <code>/query</code> 路径绑定到处理器 <code>HttpHandler</code> ，返回 <code>HttpContext</code>
<code>setExecutor(Executor executor)</code>	线程池。null = 默认单线程（同一时间只能处理 1 个请求）
<code>start()</code>	启动服务器 ，开始监听端口，不会阻塞主线程
<code>stop(int delay)</code>	关闭服务；delay = 等待多少秒结束现有请求，0 = 立刻关闭

HttpHandler

只有一个抽象方法，要重写具体业务逻辑，这就是connect后处理的地方

HttpExchange

客户端的请求信息存在exchange

- 请求相关方法（读客户端发过来的数据）

方法	作用
<code>String getRequestMethod()</code>	获取请求方法，返回字符串 <code>GET</code> / <code>POST</code> / <code>PUT</code>
<code>String getRequestURI()</code>	获取请求 URI，如 <code>/query</code>
<code>String getRequestProtocol()</code>	获取 HTTP 版本， <code>HTTP/1.1</code>
<code>InputStream getRequestBody()</code>	获取请求体输入流 ，POST 的 JSON / 表单从这里读！
<code>Headers getRequestHeaders()</code>	获取 请求头 ，可以拿到 Content-Type、User-Agent 等

- 响应相关方法（返回数据给客户端）

方法	作用
<code>Headers getResponseHeaders()</code>	拿到 响应头集合 ，设置返回给浏览器 / 客户端的头。⚠ 必须在 <code>sendResponseHeaders</code> 之前调用！
<code>sendResponseHeaders(int rCode, long responseLength)</code>	发送状态码 + 响应头（前面重点讲过）
<code>OutputStream getResponseBody()</code>	响应体输出流 ，写 JSON、文本给客户端。⚠ 必须在 <code>sendResponseHeaders</code> 之后调用！
<code>void close()</code>	关闭本次请求，释放资源

Headers

本质就是 `Map<String, List<String>>`

HttpURLConnection向客户端发送请求

方法	作用
<code>setRequestMethod("POST")</code>	设置请求方法 GET POST
<code>setRequestProperty(key, val)</code>	设置请求头
<code>setDoOutput(true)</code>	开启请求体 (POST 必须)
<code>getOutputStream()</code>	输出流, 写 JSON 请求体
<code>getResponseCode()</code>	拿到服务返回的 HTTP 状态码 200 / 404
<code>getInputStream()</code>	读取服务端返回的响应体 (成功时)
<code>getErrorStream()</code>	读取错误响应 (4xx/5xx 时)
<code>disconnect()</code>	断开连接

GET和POST的区别

POST有请求体放数据, GET的数据在网址后面, 例: `http://127.0.0.1:8080/query?expressNo=SF123456`

关于connect的步骤

```
// 1. 构造URL对象
URL url = URI.create(SERVER_URL).toURL();

// 2. 打开连接对象 (⚠此时还没建立网络TCP连接!)
HttpURLConnection connection = (HttpURLConnection) url.openConnection();

// 3. 设置请求方法 POST
connection.setRequestMethod("POST");

// 4. 设置请求头 (传JSON必须)
connection.setRequestProperty("Content-Type", "application/json");

// 5. setDoOutput(true) 【POST写body必须!】
connection.setDoOutput(true);

// 6. 获取输出流, 写入请求体JSON (这一步才真正触发网络连接)
try (OutputStreamWriter writer = new
OutputStreamWriter(connection.getOutputStream())) {
    writer.write(jsonStr);
}

// 7. 获取响应码, 拿到服务器结果
int code = connection.getResponseCode();

// 8. 判断响应码, 读取响应流
```

```
InputStream is;
if (code == 200) {
    is = connection.getInputStream();
} else {
    is = connection.getErrorStream(); // 报错的时候必须读错误流!
}
String respBody = readBody(is);

// 9. 断开连接，释放资源
connection.disconnect();
```